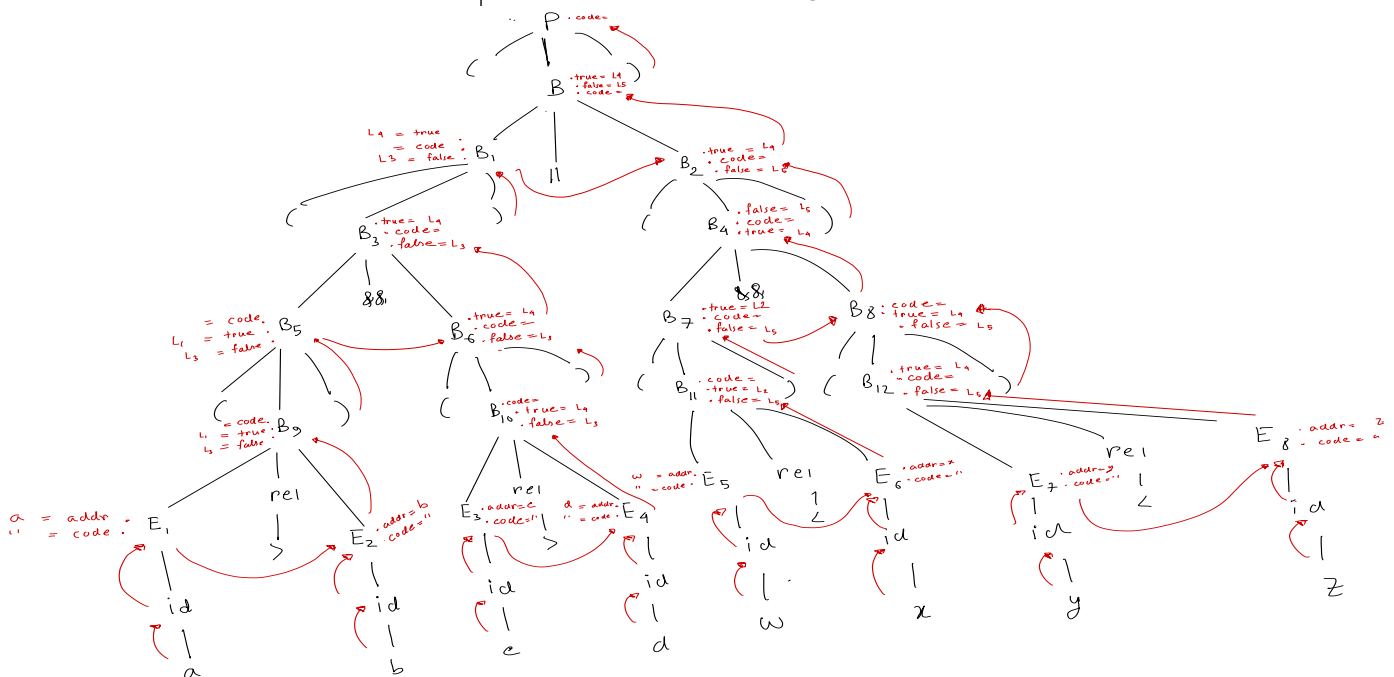


1. Consider the following **SDD**, Draw the **Annotated Parse Tree** and **Evaluate** it for the String $((a > b) \&\& (c > d)) \parallel ((w < x) \&\& (y < z))$

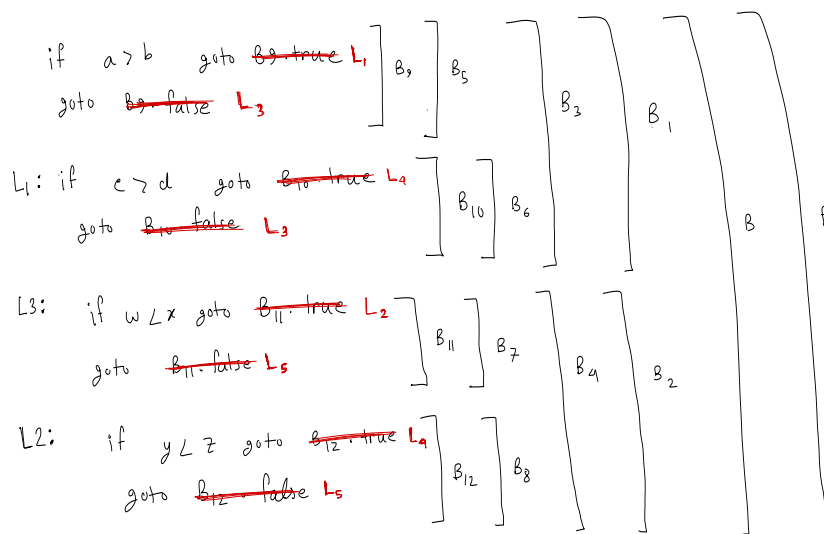
Note: Here `newLabel()` method creates a label which is 'L:' with a subscript 1, 2, 3 and so on. For example, the first time the method is called it will create L_1 ; next time it is called it will create L_2 ; and so on.

PRODUCTION	SEMANTIC RULES
$P \rightarrow (B)$	$B.true = \text{newlabel}()$ $B.false = \text{newlabel}()$ $P.code = B.code$
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = \text{newlabel}()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = \text{newlabel}()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1.true) \parallel B_2.code$
$B \rightarrow (B_1)$	$B_1.true = B.true$ $B_1.false = B.false$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel \text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$ $\parallel \text{gen('goto' } B.false)$
$E \rightarrow \text{id}$	$E.addr = \text{top.get(id.lexeme)}$ $E.code = ''$

$((a > b) \&\& (c > d)) \parallel ((w < x) \&\& (y < z))$



TAC:



L_4 : Inside Scope

L_5 : Exit Scope.

Q21

2. Consider the following SDT. Draw the **Annotated Parse Tree** and **Evaluate** the values of the attributes **type** and **width** along with variable **offset** next to the appropriate non-terminal nodes for the following String:

float [4] [5] x; record {int a; float b; } y; int z;

Afterwards draw the memory diagram along with actual location and data

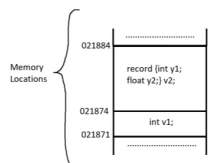
Note: Offsets are given in decimal. Memory location is also given in decimal. Every memory unit is of 1 byte. The width of int and float is given in byte in the SDT. The variable **memory_addr** refers to the starting position of the memory address where the variables will be stored.

SDT:

$P \rightarrow \{ \text{offset} = 0; \text{memory_addr} = 58342; \} D$
 $D \rightarrow T \text{ id} ; \{ \text{top.push(id.lexeme, T.type, offset)}; D_1; \text{offset} = \text{offset} + T.\text{width}; \}$
 $D \rightarrow \epsilon$
 $T \rightarrow \text{record } \{ \text{Env.push(top); top = new Env(); } D' \} \{ T.type = \text{record(top); } T.\text{width} = \text{offset}; \text{top} = \text{Env.pop(); } \text{offset} = \text{Stack.pop();} \}$
 $T \rightarrow B \{ t = B.type; w = B.width; \} C$
 $B \rightarrow \text{int}$
 $B \rightarrow \text{float}$
 $C \rightarrow \epsilon$
 $C \rightarrow [\text{num}] C_1$

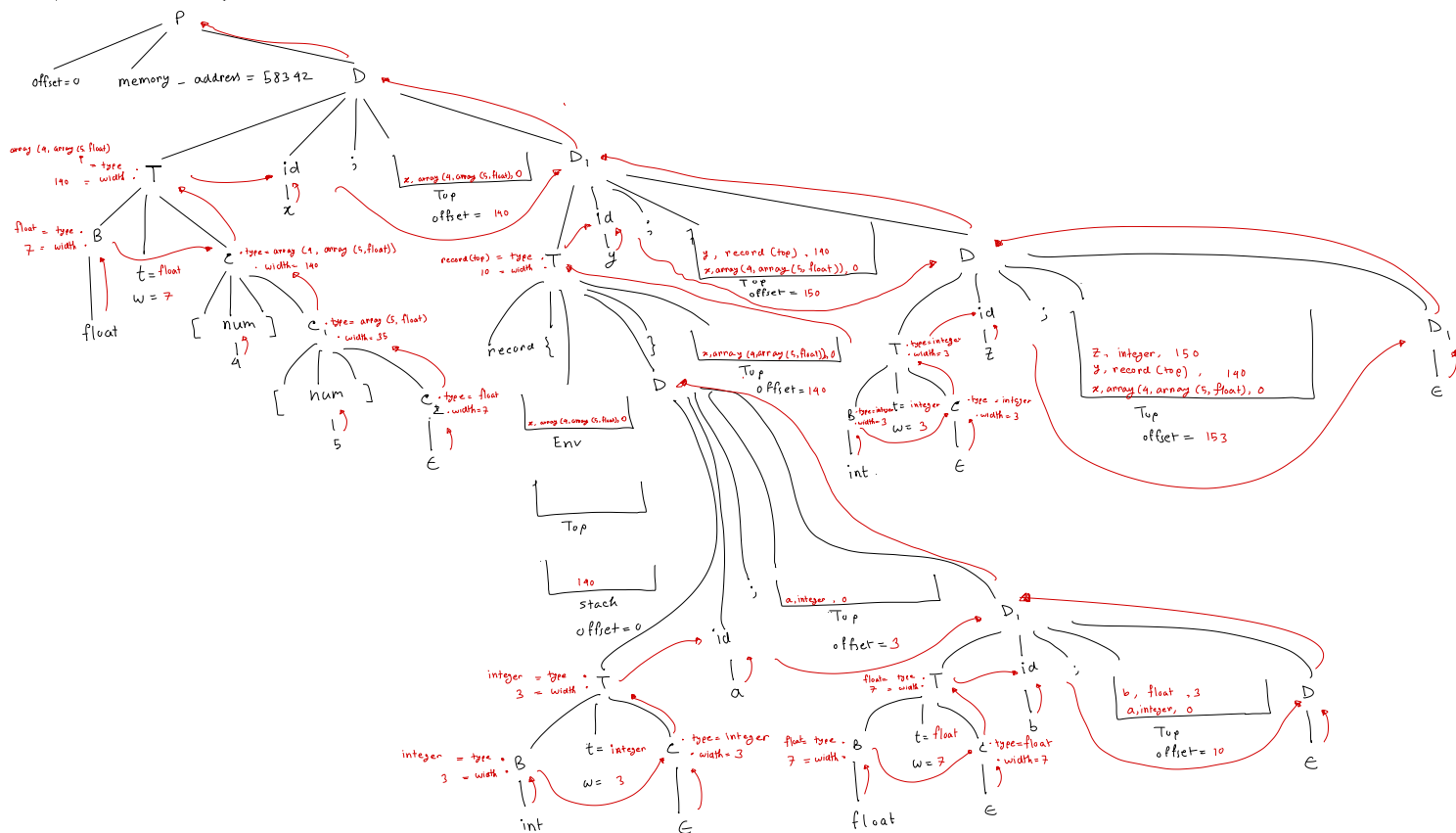
$\{ T.type = \text{record(top); } T.\text{width} = \text{offset}; \text{top} = \text{Env.pop(); } \text{offset} = \text{Stack.pop();} \}$
 $\{ T.type = C.type; T.\text{width} = C.\text{width}; \}$
 $\{ B.type = \text{integer; } B.\text{width} = 3; \}$
 $\{ B.type = \text{float; } B.\text{width} = 7; \}$
 $\{ C.type = t; C.\text{width} = w; \}$
 $\{ C.type = \text{array(num.value, } C_1.type); C.\text{width} = \text{num.value} \times C_1.\text{width}; \}$

Example of Memory Diagram for String: int v1; record {int y1; float y2; } v2; where **memory_addr** = 021871; in the SDT.



float [4] [5] x; record {int a; float b;} y; int z;

To solve this problem always keep an eye on the offset value.



58493	
58492	int z;
58482	record { int a; float b; } y;
58342	float [4] [5] x;

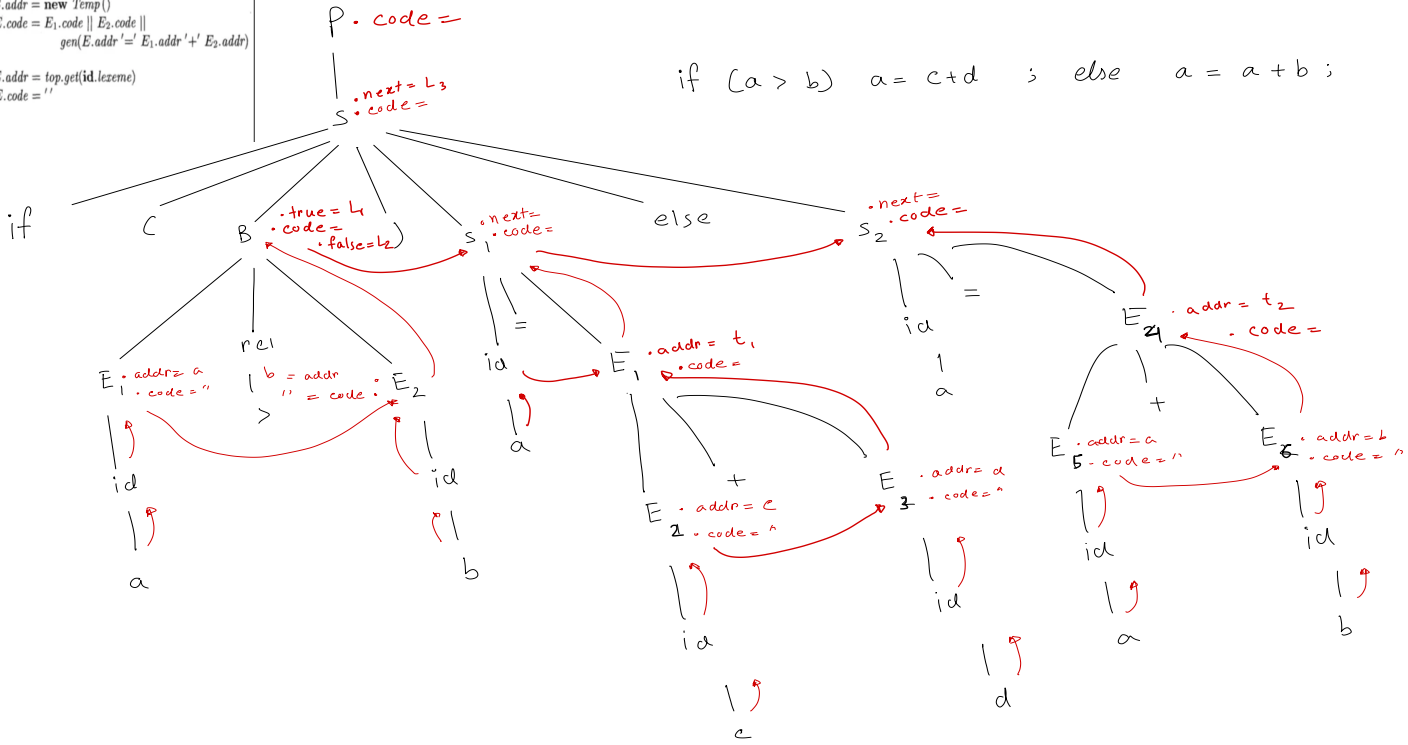
3. Consider the following SDD. Draw the Annotated Parse Tree and Evaluate it for the String: $\text{if } (a > b) \ a = c + d ; \text{ else } a = a + b ;$

Note: Here new Temp() creates an instance of compiler generated temporary variables which is 't' with a subscript 1, 2, 3 and so on. For example, the first time an instance is created it will create t_1 , next time it is called it will create t_2 and so on. Furthermore, the newLabel() method creates a label which is 'L:' with a subscript that follows similar logic as the temporary variable, for example, L_1 , L_2 , and so on.

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = \text{newLabel}()$ $P.code = S.code \parallel \text{label}(S.next)$
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $\text{gen}(\text{top.get}(\text{id.lezeme}) \neq E.addr)$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.true = \text{newLabel}()$ $B.false = \text{newLabel}()$ $S_1.next = S_2.next = S.next$ $S.code = B.code \parallel$ $\parallel \text{label}(B.true) \parallel S_1.code$ $\parallel \text{gen}(\text{'goto' } S.next)$ $\parallel \text{label}(B.false) \parallel S_2.code$
$E \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel \text{gen}(\text{'if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$ $\parallel \text{gen}(\text{'goto' } B.false)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $\text{gen}(E.addr \neq E_1.addr \neq E_2.addr)$
$\mid \text{id}$	$E.addr = \text{top.get}(\text{id.lezeme})$ $E.code = ''$

23

if (a > b) a = c + d ; else a = a + b ;



TAC:

